Name: Alhassan Abdul Waris

GitHub: AbdiWaris

Team repo: https://github.com/AbdiWaris/DE-DCIT208-GROUP-19

Personal repo: https://github.com/AbdiWaris

Project: Brainy Bites - Online Food Ordering and Management System

Youtube: https://www.youtube.com/watch?v=-M8GrF2b7M0

IA01: Personal System Request-Flow Diagram

Feature: Customer places an order (checkout.php)

Branch: ia1-AbdiWaris-checkout-request-flow · **PR:** IA1 Evidence - AbdiWaris Alhassan Abdul Waris

Request-flow explanation

(1) User device. The client confirmed that customers include students, workers and families in the area, most of whom will use smartphones. The page has a viewport meta tag and a responsive menu toggle for small screens.

(2) Browser. There is no native app yet - the client asked for a mobile-friendly website first. The customer uses the phone browser. checkout.php is server-rendered HTML. assets/js/mn.js shows the address field only when Delivery is chosen and disables the Place Order button after the first click to stop impatient double-submits.

(3) URL and route. A GET request to /checkout.php renders the order summary form. A POST request places the order. The page calls require_login() first - unauthenticated users are redirected to login.php , and users with an empty cart are redirected to cart.php .

The client confirmed customers must create accounts, which is why login is required before checkout.

(4) Protocol. HTTP over TCP. Development runs on pln HTTP (`php -S`). Any production deployment must use HTTPS because the request carries a session cookie, the customer's delivery address and order detls - all of which the client sd should be private. HTTPS is planned but not yet implemented: there is no redirect rule and the session cookie flags (`Secure` , `HttpOnly` , `SameSite`) are not set explicitly. Before the request is sent, the browser resolves the domn via DNS; no live domn exists yet.

(5) Data sent. A form-encoded POST body with six fields: `csrf_token` , `checkout_token` , `fulfilment` (pickup or delivery), `delivery_address` , `scheduled_for` (optional - the client asked for scheduled orders), and `payment_method` (cash, mobile_money or card - all three methods the client specified). The session cookie carries the PHP session ID. Item names, quantities and prices are not sent by the browser; they live server-side in `$_SESSION['cart']` .

(6) Validation. The server checks in order: logged in → cart not empty → CSRF token matches → one-time checkout token matches → `fulfilment` and `payment_method` on a whitelist → schedule is a real future datetime in `Y-m-d\TH:i` format → delivery address present if delivery chosen. Each cart line is then re-fetched from `menu_items` : unavlable items are dropped (the client asked for automatic hiding of unavlable items), the price comes from the database not the session (test TC-14), quantity is capped at 1–20, and notes are capped at 250 characters.

(7) Database. One `SELECT` on `menu_items` to confirm avlability and price. Then inside one transaction: `INSERT` into `orders` , one `INSERT` per line into `order_items` , and one `INSERT` into `payments`

with status pending . No external payment gateway is called - card and Mobile Money are recorded only (demo mode, to be integrated with a

real provider later as the client requested).

(8) Success response. On success: COMMIT , cart cleared, checkout token rotated, 302

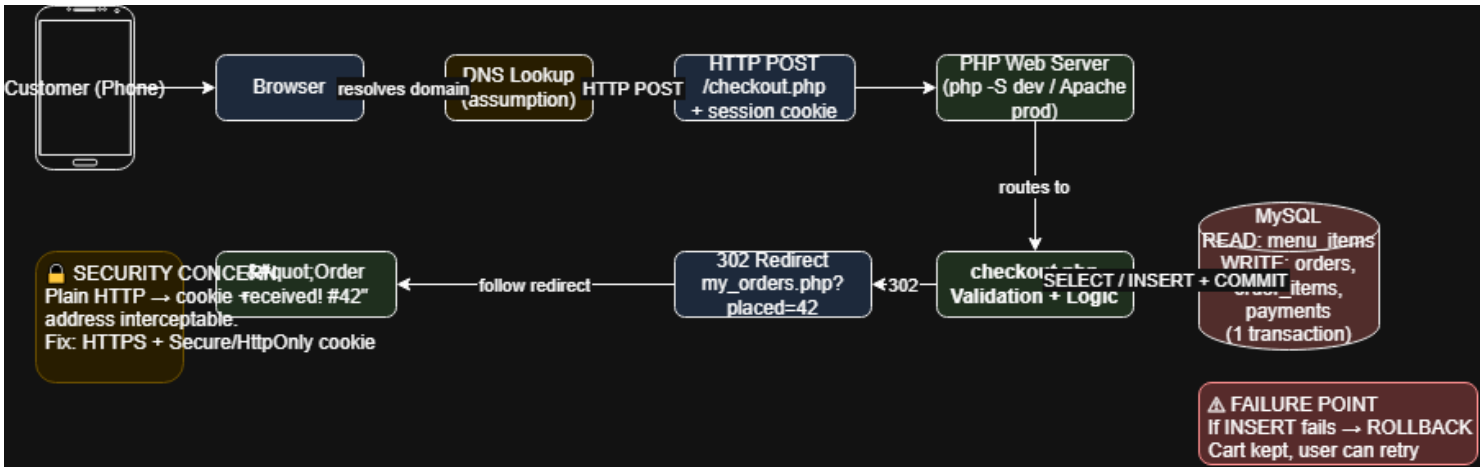
Location: my_orders.php?placed=<id> . my_orders.php returns HTML with a green "Order received" banner and the order number - satisfying the client's requirement for order tracking and receipts.

(9) Validation flure. Example: Delivery chosen with an empty address. Nothing is written to the database. The checkout page is re-rendered (HTTP 200) with the alert "Enter a delivery address." The cart and token are unchanged so the customer can correct the error and resubmit.

(10) Database fls mid-checkout. catch (Throwable) runs $pdo->rollBack() , logs the error to the server log only (never to the browser) and shows "We could not save logs the error to the server log only (never to the browser), and shows We could not save the order. Please try agn." The cart and token are preserved so the customer can retry without re-entering. If MySQL is unreachable at page load, config/database.php returns HTTP 500 for every page.

(11) Scale - reasoning only, not tested. php -S is single-process and developmentonly. Production needs Apache or Nginx with PHP-FPM. No connection pooling: each request opens its own PDO connection. The admin dashboard query (SELECT COUNT(*), SUM(...) FROM orders) scans the full table on every page load and will slow as orders grow. The client mentioned future plans for more branches, which would require load testing (e.g. k6) and possibly a caching layer before that expansion.

(12) Security and privacy concern. The client sd customer data and business data should be backed up and that only relevant staff should see relevant information. Currently, delivery addresses and session cookies travel over pln HTTP. Fix: HTTPS plus Secure , HttpOnly and SameSite=Strict cookie flags. Already in place: CSRF token on every form, password_hash for stored passwords, all SQL uses prepared statements, output uses the e() helper for escaping, and server-side re-pricing prevents price tampering.



drawn with draw.io